
Reinforcement Learning – II

CS5491: Artificial Intelligence
ZHICHAO LU

Content Credits: [Prof. Wei's CS4486 Course](#)
and [Prof. Boddeti's AI Course](#)



TODAY

Temporal–Difference Learning

Active Reinforcement Learning

Exploration vs Exploitation



RECAP: REINFORCEMENT LEARNING

Still assume a Markov decision process (MDP):

- A set of states $\mathbf{s} \in \mathcal{S}$
- A set of actions per state $\mathcal{A}$
- A model $T(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
- A reward function $R(\mathbf{s}, \mathbf{a}, \mathbf{s}')$



RECAP: REINFORCEMENT LEARNING

Still assume a Markov decision process (MDP):

- A set of states $\mathbf{s} \in \mathcal{S}$
- A set of states per state $\mathcal{A}$
- A model $\mathbf{T}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
- A reward function $\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$

Still looking for a policy $\pi(\mathbf{s})$



RECAP: REINFORCEMENT LEARNING

Still assume a Markov decision process (MDP):

- A set of states $\mathbf{s} \in \mathcal{S}$
- A set of states per state $\mathcal{A}$
- A model $\mathbf{T}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
- A reward function $\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$

Still looking for a policy $\pi(\mathbf{s})$

New twist: do not know $\mathbf{T}$ or $\mathbf{R}$



RECAP: REINFORCEMENT LEARNING

Still assume a Markov decision process (MDP):

- A set of states $\mathbf{s} \in \mathcal{S}$
- A set of states per state $\mathcal{A}$
- A model $\mathbf{T}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
- A reward function $\mathbf{R}(\mathbf{s}, \mathbf{a}, \mathbf{s}')$

Still looking for a policy $\pi(\mathbf{s})$

New twist: do not know $\mathbf{T}$ or $\mathbf{R}$

Big idea: Compute all averages over $\mathbf{T}$ using sample outcomes



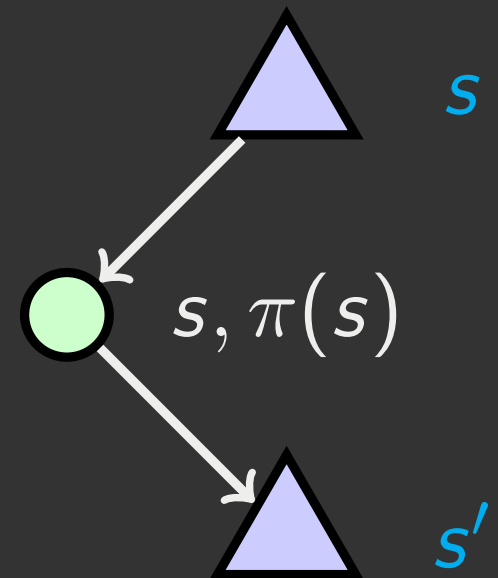
TEMPORAL-DIFFERENCE LEARNING



TEMPORAL-DIFFERENCE LEARNING

Big idea: learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often



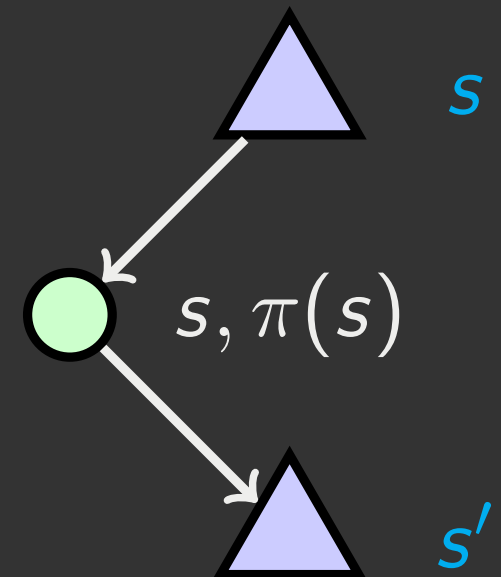
TEMPORAL-DIFFERENCE LEARNING

Big idea: learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often

Temporal difference learning of values

- Policy still fixed, still doing evaluation
- Move values toward value of whatever successor occurs: running average



TEMPORAL-DIFFERENCE LEARNING

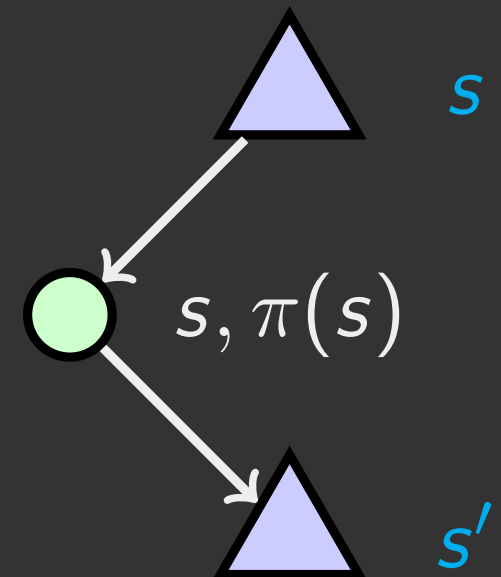
Big idea: learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often

Temporal difference learning of values

- Policy still fixed, still doing evaluation
- Move values toward value of whatever successor occurs: running average

$$\text{Sample of } V(s): \text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s') \quad (7)$$



TEMPORAL-DIFFERENCE LEARNING

Big idea: learn from every experience

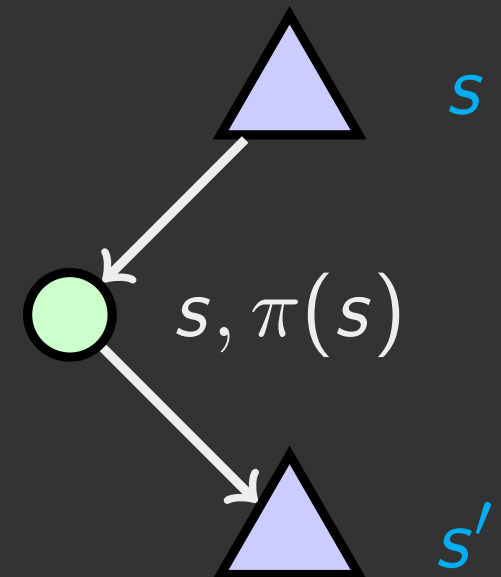
- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often

Temporal difference learning of values

- Policy still fixed, still doing evaluation
- Move values toward value of whatever successor occurs: running average

$$\text{Sample of } V(s): \text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s') \quad (10)$$

$$\text{Update to } V(s): V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + (\alpha)\text{sample} \quad (11)$$



TEMPORAL-DIFFERENCE LEARNING

Big idea: learn from every experience

- Update $V(s)$ each time we experience a transition (s, a, s', r)
- Likely outcomes s' will contribute updates more often

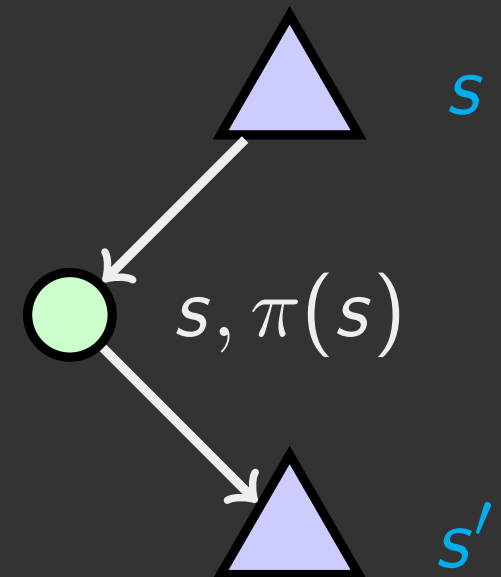
Temporal difference learning of values

- Policy still fixed, still doing evaluation
- Move values toward value of whatever successor occurs: running average

$$\text{Sample of } V(s): \text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s') \quad (13)$$

$$\text{Update to } V(s): V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + (\alpha)\text{sample} \quad (14)$$

$$\text{Same Update } V^\pi(s) = V^\pi(s) + \alpha(\text{sample} - V^\pi(s)) \quad (15)$$



EXPONENTIAL MOVING AVERAGE

Exponential moving average



EXPONENTIAL MOVING AVERAGE

Exponential moving average

- The running interpolation update:

$$\bar{x}_n = (1 - \alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$$



EXPONENTIAL MOVING AVERAGE

Exponential moving average

- The running interpolation update:

$$\bar{x}_n = (1 - \alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$$

- Makes recent samples more important:

$$\bar{x}_n = \frac{x_n + (1 - \alpha) \cdot x_{n-1} + (1 - \alpha)^2 \cdot x_{n-2} + \dots}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots}$$



EXPONENTIAL MOVING AVERAGE

Exponential moving average

- The running interpolation update:

$$\bar{x}_n = (1 - \alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$$

- Makes recent samples more important:

$$\bar{x}_n = \frac{x_n + (1 - \alpha) \cdot x_{n-1} + (1 - \alpha)^2 \cdot x_{n-2} + \dots}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots}$$

- Forgets about the past (distant past values were wrong anyway)



EXPONENTIAL MOVING AVERAGE

Exponential moving average

- The running interpolation update:

$$\bar{x}_n = (1 - \alpha) \cdot \bar{x}_{n-1} + \alpha \cdot x_n$$

- Makes recent samples more important:

$$\bar{x}_n = \frac{x_n + (1 - \alpha) \cdot x_{n-1} + (1 - \alpha)^2 \cdot x_{n-2} + \dots}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots}$$

- Forgets about the past (distant past values were wrong anyway)

Decreasing learning rate (α) can give converging averages



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

States

	A	
B	C	D
	E	

Assume $\gamma = 1$, $\alpha = 0.5$



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

States

	A	
B	C	D
	E	

	0	
0	0	8
	0	

Assume $\gamma = 1$, $\alpha = 0.5$



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

Observed Transitions

States

	A	
B	C	D
	E	

	0	
0	0	8
	0	

B, east, C, -2

Assume $\gamma = 1$, $\alpha = 0.5$

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma V^\pi(s')]$$



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

Observed Transitions

States

	A	
B	C	D
	E	

	0	
0	0	8
	0	

	0	
-1	0	8
	0	

B,east,C,-2

Assume $\gamma = 1$, $\alpha = 0.5$

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma V^\pi(s')]$$



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

Observed Transitions

States

	A	
B	C	D
	E	

Assume $\gamma = 1$, $\alpha = 0.5$

B,east,C,-2

	0	
0	0	8
	0	

C,east,D,-2

	0	
-1	0	8
	0	

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma V^\pi(s')]$$



EXAMPLE: TEMPORAL-DIFFERENCE LEARNING

Observed Transitions

States

	A	
B	C	D
	E	

Assume $\gamma = 1$, $\alpha = 0.5$

B,east,C,-2

	0	
0	0	8
	0	

C,east,D,-2

	0	
-1	0	8
	0	

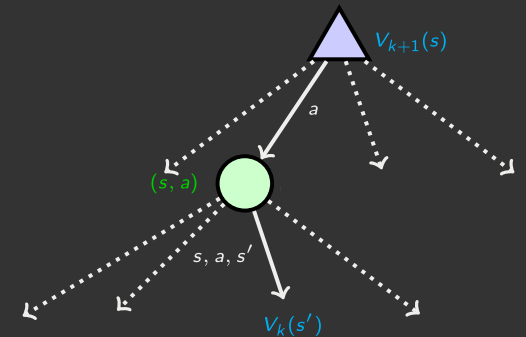
	0	
-1	3	8
	0	

$$V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma V^\pi(s')]$$



PROBLEMS WITH TD VALUE LEARNING

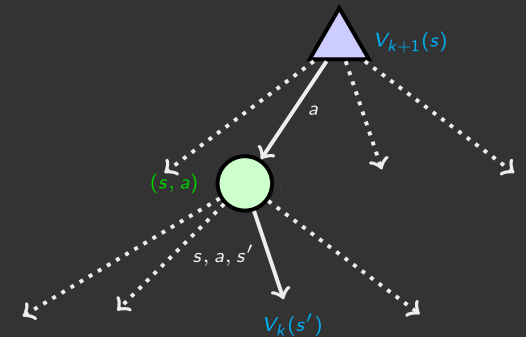
TD value learning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages



PROBLEMS WITH TD VALUE LEARNING

TD value learning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages

However, if we want to turn values into a (new) policy, we are sunk:



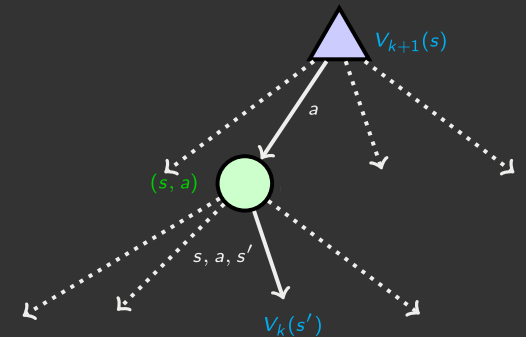
PROBLEMS WITH TD VALUE LEARNING

TD value learning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages

However, if we want to turn values into a (new) policy, we are sunk:

$$\pi(s) = \operatorname{argmax}_{\mathbf{a}} Q(s, a)$$

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$



PROBLEMS WITH TD VALUE LEARNING

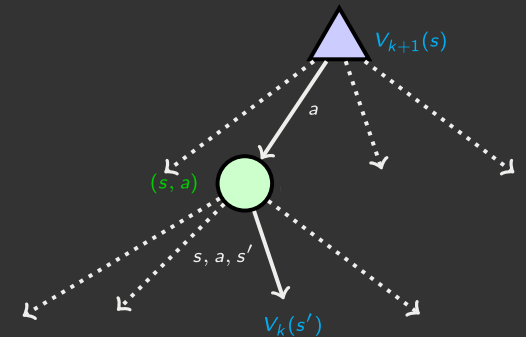
TD value learning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages

$$\pi(s) = \underset{\mathbf{a}}{\operatorname{argmax}} Q(s, a)$$

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

Idea: learn Q -values, not values

Makes action selection model-free too



ACTIVE REINFORCEMENT LEARNING



ACTIVE REINFORCEMENT LEARNING

Full reinforcement learning: optimal policies (like value iteration)



ACTIVE REINFORCEMENT LEARNING

Full reinforcement learning: optimal policies (like value iteration)

- You do not know the transitions $T(s, a, s')$
- You do not know the rewards $R(s, a, s')$
- You choose the actions now
- Goal: learn the optimal policy / values



ACTIVE REINFORCEMENT LEARNING

Full reinforcement learning: optimal policies (like value iteration)

- You do not know the transitions $T(s, a, s')$
- You do not know the rewards $R(s, a, s')$
- You choose the actions now
- Goal: learn the optimal policy / values

In this case:



ACTIVE REINFORCEMENT LEARNING

Full reinforcement learning: optimal policies (like value iteration)

- You do not know the transitions $T(s, a, s')$
- You do not know the rewards $R(s, a, s')$
- You choose the actions now
- Goal: learn the optimal policy / values

In this case:

- Learner makes choices.
- Fundamental trade-off: exploration vs. exploitation
- This is NOT off-line planning. You actually take actions in the world and find out what happens . . .



DETOUR: Q -VALUE ITERATION

Value iteration: find successive (depth-limited) values

- Start with $V_0(\mathbf{s}) = \mathbf{0}$, which we know is right
- Given V_k , calculate the depth $k + 1$ values for all states:



DETOUR: Q -VALUE ITERATION

Value iteration: find successive (depth-limited) values

- Start with $V_0(s) = 0$, which we know is right
- Given V_k , calculate the depth $k + 1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$



DETOUR: Q -VALUE ITERATION

Value iteration: find successive (depth-limited) values

- Start with $V_0(s) = 0$, which we know is right
- Given V_k , calculate the depth $k + 1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

But Q -values are more useful, so compute them instead

- Start with $Q_0(s, a) = 0$ which we know is right
- Given Q_k , calculate the depth $k + 1$ q -values for all q -states:



DETOUR: Q -VALUE ITERATION

Value iteration: find successive (depth-limited) values

- Start with $V_0(s) = 0$, which we know is right
- Given V_k , calculate the depth $k + 1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

But Q -values are more useful, so compute them instead

- Start with $Q_0(s, a) = 0$ which we know is right
- Given Q_k , calculate the depth $k + 1$ q -values for all q -states:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$$



Q -LEARNING

Q -Learning: sample-based Q -value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (16)$$



Q -LEARNING

Q -Learning: sample-based Q -value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (19)$$

Learn $Q(s, a)$ values as you go



Q -LEARNING

Q -Learning: sample-based Q -value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (22)$$

Learn $Q(s, a)$ values as you go

- Receive a sample (s, a, s', r)

Q -LEARNING

Q -Learning: sample-based Q -value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (25)$$

Learn $Q(s, a)$ values as you go

- Receive a sample (s, a, s', r)
- Consider your old estimate: $Q(s, a)$

Q-LEARNING

Q-Learning: sample-based Q-value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (28)$$

Learn $Q(s, a)$ values as you go

- Receive a sample (s, a, s', r)
- Consider your old estimate: $Q(s, a)$
- Consider your new sample estimate:

$$sample = R(s, a, s') + \gamma \max_{a'} Q(s', a') \quad (29)$$

Q-LEARNING

Q-Learning: sample-based Q-value iteration

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')] \quad (31)$$

Learn $Q(s, a)$ values as you go

- Receive a sample (s, a, s', r)
- Consider your old estimate: $Q(s, a)$
- Consider your new sample estimate:

$$sample = R(s, a, s') + \gamma \max_{a'} Q(s', a') \quad (32)$$

- Incorporate the new estimate into a running average:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha)sample \quad (33)$$

Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy -- even if you are acting sub-optimally.



Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy -- even if you are acting sub-optimally.

This is called off-policy learning.



Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy -- even if you are acting sub-optimally.

This is called off-policy learning.

Caveats:

- You have to explore enough



Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy — even if you are acting sub-optimally.

This is called off-policy learning.

Caveats:

- You have to explore enough
- You have to eventually make the learning rate small enough

Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy — even if you are acting sub-optimally.

This is called off-policy learning.

Caveats:

- You have to explore enough
- You have to eventually make the learning rate small enough
- ... but not decrease it too quickly



Q -LEARNING PROPERTIES

Amazing result: Q -learning converges to optimal policy — even if you are acting sub-optimally.

This is called off-policy learning.

Caveats:

- You have to explore enough
- You have to eventually make the learning rate small enough
- ... but not decrease it too quickly
- Basically, in the limit, it does not matter how you select actions.



THE STORY SO FAR: MDPS VS RL



THE STORY SO FAR: MDPS VS RL

Known MDP: Offline Solution

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Value / policy iteration

Policy evaluation

THE STORY SO FAR: MDPS VS RL

Known MDP: Offline Solution

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Value / policy iteration

Policy evaluation

Unknown MDP: Model-Based

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

VI/PI on approx. MDP

PE on approx. MDP



THE STORY SO FAR: MDPS VS RL

Known MDP: Offline Solution

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Value / policy iteration

Policy evaluation

Unknown MDP: Model-Based

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

VI/PI on approx. MDP

PE on approx. MDP

Unknown MDP: Model-Free

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Q-Learning

Value Learning



EXPLORATION VS EXPLOITATION



HOW TO EXPLORE?

Several schemes for forcing exploration



HOW TO EXPLORE?

Several schemes for forcing exploration

- Simplest: random actions (ϵ -greedy)



HOW TO EXPLORE?

Several schemes for forcing exploration

- Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1 - \epsilon$, act on current policy



HOW TO EXPLORE?

Several schemes for forcing exploration

- Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1 - \epsilon$, act on current policy
- Problems with random actions?



HOW TO EXPLORE?

Several schemes for forcing exploration

- Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1 - \epsilon$, act on current policy
- Problems with random actions?
 - You do eventually explore the space, but keep thrashing around once learning is done
 - One solution: lower ϵ over time
 - Another solution: exploration functions



EXPLORATION FUNCTIONS

When to explore?



EXPLORATION FUNCTIONS

When to explore?

- Random actions: explore a fixed amount
- Better idea: explore areas whose badness is not (yet) established, eventually stop exploring



EXPLORATION FUNCTIONS

When to explore?

- Random actions: explore a fixed amount
- Better idea: explore areas whose badness is not (yet) established, eventually stop exploring

Exploration function

- Takes a value estimate u and a visit count n , and returns an optimistic utility, e.g.

$$f(u, n) = u + \frac{k}{n+1}$$



EXPLORATION FUNCTIONS

When to explore?

- Random actions: explore a fixed amount
- Better idea: explore areas whose badness is not (yet) established, eventually stop exploring

Exploration function

- Takes a value estimate u and a visit count n , and returns an optimistic utility, e.g.

$$f(u, n) = u + \frac{k}{n+1}$$

$$\text{Regular } Q\text{-Update} : Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} Q(s', a')] \quad (40)$$

$$\text{Modified } Q\text{-Update} : Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} f(Q(s', a'), N(s', a'))] \quad (41)$$



EXPLORATION FUNCTIONS

When to explore?

- Random actions: explore a fixed amount
- Better idea: explore areas whose badness is not (yet) established, eventually stop exploring

Exploration function

- Takes a value estimate u and a visit count n , and returns an optimistic utility, e.g.

$$f(u, n) = u + \frac{k}{n+1}$$

$$\text{Regular } Q\text{-Update} : Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} Q(s', a')] \quad (42)$$

$$\text{Modified } Q\text{-Update} : Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[R(s, a, s') + \gamma \max_{a'} f(Q(s', a'), N(s', a'))] \quad (43)$$

- Note: this propagates the "bonus" back to states that lead to unknown states as well.



Q & A



XKCD

